

LAPORAN PRAKTIKUM PBO 5

Abstract Class & Interface



Disusun Oleh:
Ananda Firdaus (251511003)
Jurusan Teknik Komputer dan Informatika
Rabu, 7 Oktober 2026

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2025

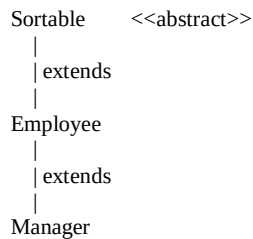
Repo: <https://github.com/FirdausSinau/PBO>

BAGIAN A - Employee, Manager, dan Sortable

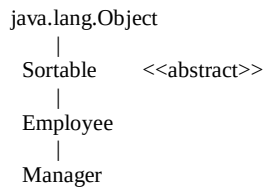
A.1 Tujuan

Praktikum pertama membangun kembali inheritance dari Employee ke Manager, kemudian menambahkan sebuah abstract class bernama Sortable. Tujuannya melihat bagaimana abstract method membentuk kewajiban pada subclass, dan bagaimana inheritance chain bekerja tanpa menulis extends dua kali.

A.2 Diagram Hierarchy



Rantai lengkap sampai akar Java:



A.3 Code Inti

Sortable.java

```
public abstract class Sortable {
    public abstract int compare(Sortable other);
}
```

Hanya berisi satu method tanpa body. Tidak ada field, tidak ada method konkret, tidak ada constructor.

Employee.java

```
public class Employee extends Sortable {
    private String name;
    private double salary;
    private int hireDay;
    private int hireMonth;
    private int hireYear;

    public Employee(String name, double salary,
                    int day, int month, int year) {
        this.name = name;
        this.salary = salary;
        this.hireDay = day;
        this.hireMonth = month;
        this.hireYear = year;
    }
}
```

```

    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }

    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }

    public int hireYear() {
        return hireYear;
    }

    public double getSalary() {
        return salary;
    }

    @Override
    public int compare(Sortable other) {
        Employee otherEmployee = (Employee) other;
        if (getSalary() < otherEmployee.getSalary()) return -1;
        if (getSalary() > otherEmployee.getSalary()) return 1;
        return 0;
    }
}

```

Manager.java

```

import java.util.Calendar;
import java.util.GregorianCalendar;

public class Manager extends Employee {
    private String secretaryName;

    public Manager(String name, double salary, int day, int month, int year) {
        super(name, salary, day, month, year);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        GregorianCalendar today = new GregorianCalendar();
        int currentYear = today.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());
        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}

```

A.4 Output EmployeeTest

=== 1. SEBELUM kenaikan ===

Antonio Rossi 2000000.0 1989
 Maria Bianchi 2500000.0 1991
 Isabel Vidal 3000000.0 1993

=== 2. SESUDAH kenaikan 5% ===

Antonio Rossi 2100000.0 1989
 Maria Bianchi 3062500.0 1991
 Isabel Vidal 3150000.0 1993

```

=== 3. Uji compare() ===
staff[0].compare(staff[1]) = -1
staff[1].compare(staff[2]) = -1
staff[2].compare(staff[0]) = 1
staff[0].compare(staff[0]) = 0

```

Caption: staff[1] adalah object Manager, sedangkan staff[0] dan staff[2] adalah Employee biasa. Ketiganya dipanggil dengan e.raiseSalary(5) yang identik, tetapi hasilnya berbeda.

Perhitungan kenaikan (tahun berjalan 2026):

Object	Tipe asli	Kenaikan	Sebelum	Sesudah
Antonio Rossi	Employee	5%	2.000.000	2.100.000
Maria Bianchi	Manager	5% + 17.5% bonus = 22.5%	2.500.000	3.062.500
Isabel Vidal	Employee	5%	3.000.000	3.150.000

Perhitungan bonus Manager:

```

bonus = 0.5 * (currentYear - hireYear()) = 0.5 * (2026 - 1991) = 0.5 * 35 = 17.5
total = 5 + 17.5 = 22.5
2.500.000 * (1 + 22.5/100) = 2.500.000 * 1.225 = 3.062.500

```

Catatan: angka bonus bergantung pada tahun saat program dijalankan (GregorianCalendar), jadi hasilnya berbeda dari contoh di modul yang ditulis tahun 2020.

A.5 Bukti Abstract Class Tidak Dapat Diinstansiasi

Eksperimen: Sortable s = new Sortable();

```

error: Sortable is abstract; cannot be instantiated
    Sortable s = new Sortable();
                  ^
1 error

```

Caption: Sortable hanya berisi kewajiban compare(...) dan tidak memiliki bentuk object konkret. Compiler menolak pembuatan object-nya. Ini masuk akal karena isi Sortable hanya satu method tanpa body - kalau objectnya dibuat, memanggil compare() tidak akan melakukan apa pun.

A.6 Bukti Employee Wajib Mengimplementasikan compare()

Eksperimen: Employee extends Sortable **tanpa** menambahkan compare().

```

error: Employee is not abstract and does not override abstract method compare(Sortable) in
Sortable
public class Employee extends Sortable {
    ^
1 error

```

Setelah compare() ditambahkan, compile kembali **bersih**.

Caption: Compiler memberikan dua pilihan - isi compare(), atau deklarasikan Employee juga sebagai abstract (kalau ingin menunda). Yang tidak mungkin adalah membiarkannya kosong sambil tetap konkret.

A.7 Hasil compare()

Aturan yang dipakai:

```

if (getSalary() < otherEmployee.getSalary()) return -1;

```

```
if (getSalary() > otherEmployee.getSalary()) return 1;
return 0;
```

Hasil (setelah kenaikan salary):

Panggilan	Perbandingan	Hasil
staff[0].compare(staff[1])	2.100.000 vs 3.062.500	-1
staff[1].compare(staff[2])	3.062.500 vs 3.150.000	-1
staff[2].compare(staff[0])	3.150.000 vs 2.100.000	1
staff[0].compare(staff[0])	2.100.000 vs dirinya sendiri	0

Arti setiap nilai: -1 berarti object pemanggil lebih kecil, 1 berarti lebih besar, dan 0 berarti sama.

A.8 Bukti Inheritance Chain Manager

Eksperimen: Manager m = new Manager(...) dan Employee e = new Employee(...), lalu m.compare(e).

```
m.compare(e) = -1
```

Manager TIDAK menulis 'extends Sortable'
 Manager TIDAK menulis compare()
 Tapi m instanceof Sortable = true

Rantai class (naik dari Manager):
 Manager
 Employee
 Sortable <<abstract>>
 java.lang.Object

Caption: Manager tidak menulis extends Sortable maupun compare(), tetapi tetap memilikinya. Warisan bersifat **transitif** - kemampuan Sortable mengalir turun lewat Employee. Seperti cucu yang mewarisi sifat kakek.

A.9 Batas extends - Java Menolak Multiple Inheritance

Eksperimen: public class Manager extends Employee extends Sortable { }

```
error: '{' expected
public class Manager extends Employee extends Sortable {
                                     ^
1 error
```

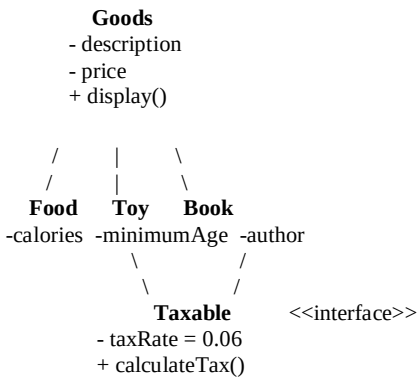
Caption: Compiler sudah selesai membaca extends Employee dan mengharapkan {, bukan extends lagi. Java hanya mengizinkan **satu** extends untuk class. Ini yang disebut multiple inheritance, dan Java tidak mendukungnya. Jalan keluarnya adalah implements (lihat Bagian B).

BAGIAN B - Goods dan Taxable

B.1 Tujuan

Blok kedua menggunakan hierarchy Goods untuk membedakan dua jenis hubungan: Food, Toy, dan Book adalah jenis Goods; sementara hanya Toy dan Book yang memiliki kontrak Taxable.

B.2 Diagram Hierarchy + Interface



Perhatikan: Food **tidak** terhubung ke Taxable.

B.3 Code Inti

Goods.java

```
public class Goods {
    private String description;
    private double price;

    public Goods(String description, double price) {
        this.description = description;
        this.price = price;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public void display() {
        System.out.println("Description : " + description);
        System.out.println("Price : " + price);
    }
}
```

Food.java

```
public class Food extends Goods {
    private int calories;

    public Food(String description, double price, int calories) {
        super(description, price);
    }
}
```

```

        this.calories = calories;
    }

    public int getCalories() {
        return calories;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Calories    : " + calories);
    }
}

```

Taxable.java

```

public interface Taxable {
    double taxRate = 0.06;
    double calculateTax();
}

```

Hanya dua baris. `taxRate` otomatis menjadi public static final (konstanta), dan `calculateTax()` otomatis public abstract walaupun tidak ditulis.

Toy.java

```

public class Toy extends Goods implements Taxable {
    private int minimumAge;

    public Toy(String description, double price, int minimumAge) {
        super(description, price);
        this.minimumAge = minimumAge;
    }

    public int getMinimumAge() {
        return minimumAge;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Minimum age : " + minimumAge);
        System.out.println("Tax        : " + calculateTax());
    }
}

```

Satu baris deklarasi memuat dua hubungan:

- `public class Toy extends Goods` berarti "Toy adalah subclass dari Goods"
- `implements Taxable`

Book.java

```

public class Book extends Goods implements Taxable {
    private String author;

    public Book(String description, double price, String author) {

```

```

        super(description, price);
        this.author = author;
    }

    public String getAuthor() {
        return author;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Author      : " + author);
        System.out.println("Tax      : " + calculateTax());
    }
}

```

Toy dan Book punya parent yang sama dan kontrak yang sama, tetapi state spesifiknya berbeda (minimumAge vs author).

B.4 Output GoodsTest

Description : Bread
Price : 15000.0
Calories : 250

Description : Robot
Price : 120000.0
Minimum age : 8
Tax : 7200.0

Description : Java Basics
Price : 90000.0
Author : A. Programmer
Tax : 5400.0

Toy tax : 7200.0
Book tax : 5400.0

=== Bonus: yang bisa dan tidak bisa jadi Taxable ===
Taxable[] berisi 2 object:
7200.0
5400.0

Food TIDAK bisa masuk Taxable[]:
Taxable t = food; -> compile error

Caption: Food tidak mencetak baris Tax karena tidak implements Taxable. Toy dan Book mencetak Tax karena ada.

B.5 Hasil Perhitungan Tax

Asumsi operasional dari modul: pajak = price * taxRate.

Object	Price	taxRate	Tax
Toy "Robot"	120.000	0.06	7.200,0
Book "Java Basics"	90.000	0.06	5.400,0

Perhitungan:

Toy : $120.000 * 0.06 = 7.200$
Book : $90.000 * 0.06 = 5.400$

B.6 Bukti Contract Interface Ketika calculateTax() Dihapus

Eksperimen: calculateTax() dihapus dari Toy, tetapi implements Taxable tetap dipertahankan.

```
error: Toy is not abstract and does not override abstract method calculateTax() in Taxable
public class Toy extends Goods implements Taxable {
    ^

error: cannot find symbol
2 errors
```

Caption: Ada dua error. Yang pertama karena Toy melanggar kontrak. Yang kedua karena display() di Toy memanggil calculateTax() yang sudah hilang.

B.7 Bukti Food Bukan Taxable

Eksperimen: Taxable t = food;

```
error: incompatible types: Food cannot be converted to Taxable
    Taxable t = food;
                ^

1 error
```

Caption: "tipe tidak cocok", Food tidak implementasi Taxable, sehingga ditolak sejak awal.

B.8 Bukti taxRate adalah Konstanta

Eksperimen tambahan: Taxable.taxRate = 0.10;

```
error: cannot assign a value to static final variable taxRate
    Taxable.taxRate = 0.10;
                ^

1 error
```

Caption: Field di interface otomatis public static final. Tidak perlu ditulis, dan tidak bisa diubah. Ini jadi sebabnya kenapa interface hanya boleh memuat konstanta, bukan variabel.

Sebagai perbandingan, Taxable.taxRate bisa dibaca langsung tanpa membuat object:

```
Taxable.taxRate = 0.06
```

B.9 Perbandingan Sortable (abstract class) dan Taxable (interface)

Tabel ini disusun berdasarkan pengalaman menjalankan kode, bukan dari definisi.

Pertanyaan	Sortable (abstract class)	Taxable (interface)
Dapat dibuat object langsung?	Tidak	Tidak
Pesan error saat dicoba	Sortable is abstract; cannot be instantiated	Taxable is abstract; cannot be instantiated (identik)
Menggunakan extends atau implements?	extends	implements
Dapat mempunyai implementasi method konkret?	Ya	Tidak, interface abstract methods cannot have body
Dapat mempunyai data member?	Ya, field biasa	Tidak, hanya konstanta (public static final)

Peran utamanya pada kasus praktikum?	Menetapkan kewajiban compare(...)	Menetapkan kontrak calculateTax()
Apakah class dapat mempunyai parent class lain sekaligus?	Tidak, hanya 1	Ya, extends Goods + implements Taxable

Kesimpulan: keduanya sama-sama abstract, dan keduanya sama-sama memaksa anak mengisi. Bedanya:

- Abstract class: bisa diisi methodnya, bisa juga tidak.
- Interface: semua method kosong, jadi Anak nya wajib membuat isinya.

Dan perbedaan yang paling menentukan: extends hanya boleh satu, sedangkan implements boleh banyak.

TABEL SKENARIO UJI T5-01 s.d. T5-08

ID	Skenario	Expected Result	Bukti	Status
T5-01	new Sortable() dicoba	Compiler menolak instansiasi abstract class	error: Sortable is abstract; cannot be instantiated	PASS
T5-02	Employee extends Sortable tanpa compare()	Compiler menuntut implementasi abstract method	error: Employee is not abstract and does not override abstract method compare(Sortable) in Sortable	PASS
T5-03	Employee mengimplementasikan compare()	Hasil perbandingan - 1/0/1 sesuai salary	staff[0].compare(staff[1])=-1, staff[1].compare(staff[2])=-1, staff[2].compare(staff[0])=1, staff[0].compare(staff[0])=0	PASS
T5-04	Manager hanya extends Employee	Manager tetap memiliki compare() melalui inheritance chain	m.compare(e)=-1 dan m instanceof Sortable=true, padahal Manager.java tidak menulis extends Sortable maupun compare()	PASS
T5-05	Manager extends Employee extends Sortable	Compiler menolak multiple extends class	error: '{' expected dengan penunjuk tepat di extends kedua	PASS
T5-06	Toy/Book implements Taxable	calculateTax() dapat dipanggil	toy.calculateTax()=7200.0, book.calculateTax()=5400.0	PASS
T5-07	calculateTax() dihapus dari Toy	Compiler menolak class	error: Toy is not abstract and does not override	PASS

		konkret yang tidak memenuhi contract	abstract method calculateTax() in Taxable	
T5-08	Food diuji sebagai Goods	State/behavior Goods tersedia melalui inheritance	Goods g = new Food(...) lalu g.display() mencetak Description, Price, Calories; g.getPrice()=15000.0	PASS

Definition of Done

#	Pernyataan	Status
1	Semua Step 1-18 telah dicoba, bukan hanya code final disalin	Done , termasuk 7 eksperimen error
2	Minimal dua bukti error eksperimen tersimpan	Done , 7 bukti: A.5, A.6, A.9, B.6, B.7, B.8
3	Saya dapat menjelaskan inheritance chain Sortable <- Employee <- Manager	Done (A.8)
4	Saya dapat menjelaskan mengapa Employee harus mengimplementasikan compare()	Done (A.6)
5	Saya dapat menunjukkan perbedaan extends dan implements dari code	Done (B.3, B.9)
6	Saya dapat menjelaskan mengapa Toy dan Book Taxable sedangkan Food tidak	Done (B.5, Refleksi 6)
7	Project final compile dan seluruh skenario utama PASS	Done , 8/8 PASS
8	PDF dan source siap dikumpulkan	Done

Self Assessment

Pernyataan	Skor 1-5
Saya memahami mengapa abstract class tidak dapat diinstansiasi.	4
Saya dapat membuat abstract method dan implementasinya di subclass.	4
Saya memahami inheritance chain tanpa menulis extends lebih dari satu class.	4
Saya dapat membuat interface dan class yang implements interface tersebut.	4
Saya memahami bahwa implements berarti memenuhi contract method.	4
Saya dapat membedakan kapan konsep lebih cocok menjadi abstract class dan kapan menjadi interface.	4

Refleksi

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Karena Sortable tidak perlu menjadi object, dia seperti menjadi syarat.

Gunanya supaya semua yang "bisa dibandingkan" punya method compare() dengan nama dan bentuk yang sama. Jadi object dari class berbeda seperti diperlakukan seragam: saya bisa membuat Sortable[] dan memanggil compare() dalam satu loop.

Tanpa Sortable, tidak ada yang menjadikan semua class (yang membutuhkan) punya compare(). Kalau buat Loop sendiri, compiler menolak karena tidak ada tipe dasar compare() nya.

2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Compile error:

```
error: Employee is not abstract and does not override abstract method compare(Sortable) in Sortable
```

Java memberi dua pilihan: isi compare(), atau jadikan Employee juga abstract kalau ingin menunda. Yang tidak bisa: dibiarkan kosong sambil tetap konkret.

Pelajarannya: abstract method wajib diisi oleh si anak/subclass nya.

3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Manager extends Employee, Employee extends Sortable. Jadi secara tidak langsung Manager juga extends Sortable.

4. Mengapa class Manager extends Employee extends Sortable tidak diperlukan dan tidak valid di Java?

Tidak diperlukan karena Manager sudah mendapat compare().

Tidak valid karena Java tidak punya sintaksnya. extends hanya boleh satu. Errornya:

```
error: '{' expected
public class Manager extends Employee extends Sortable {
                        ^
```

Maka dari itu menggunakan implements.

5. Apa yang dijanjikan Toy ketika menulis implements Taxable?

Toy berjanji akan menyediakan method calculateTax() (public double calculateTax()), dan dipanggil lewat referensi Taxable.

6. Mengapa Goods lebih tepat menjadi superclass, sedangkan Taxable lebih tepat menjadi interface?

Goods jadi superclass karena Food, Toy, dan Book benar-benar "adalah sebuah" Goods (atau is-a isitlahnya). Ketiganya butuh data yang sama (description, price) dan behavior yang sama (display()). Jadi ada warisan data + behavior.

Taxable jadi interface karena tiga alasan:

1. Tidak semua anak punya (Food tidak punya).
2. Tidak membawa data apa pun.
3. Kalau dijadikan abstract class, Toy tidak akan bisa extends Goods lagi, karena extends cuma boleh satu. Toy butuh dua-duanya.

Jadi pembagiannya: yang identitas + data jadi superclass, yang kemampuan tambahan jadi interface.

7. Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

Ada dua bukti:

- Bukti pertama, yang berjanji harus menepati. Menghapus calculateTax() dari Toy sementara implements Taxable tetap:

error: Toy is not abstract and does not override abstract method calculateTax() in Taxable

- Bukti kedua, yang tidak berjanji tidak bisa ikut. Memaksa Food menjadi Taxable:

error: incompatible types: Food cannot be converted to Taxable

Yang pertama soal **method kurang**. Yang kedua soal **tipe tidak cocok**, Food tidak pernah menandatangani kontrak, jadi ditolak sejak awal.

8. Lengkapi: "Abstract class membantu saya ketika ..., sedangkan interface membantu saya ketika ..."

Abstract class membantu saya ketika saya punya sekelompok class yang sama-sama "is-a" sesuatu dan berbagi data, sedangkan Interface membantu saya ketika yang saya butuhkan cuma kemampuan, tanpa data, dan kemampuan itu tidak dimiliki semua anak atau ketika saya perlu menempelkan lebih dari satu kemampuan ke satu class.